

Title: Enhancing Recommender Systems Using Deep Learning and Variational Techniques

Submitted by:

Bandi Bhanusri

Roll No:322103383008

BACHELOR OF TECHNOLOGY

In

Computer Science & Data Science

GAYATRI VIDHYA PARISHAD COLLEGE OF ENGINEERING



**COLLEGE OF ENGINEERING
(AUTONOMOUS)**

Internship Guide:

Dr. Venkateshwara Rao Kagita

Department of CSE (AI&ML)

National Institute of Technology, Warangal



Certificate:

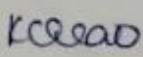
	NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL- 506004, TELANGANA, INDIA An Institute of National Importance Under MHRD, Govt. of India	
Prof. Venkateswara Rao Kagita Department of CSE		Ph. No: +91-9966456713 Email: venkat.kagita@nitw.ac.in

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that **Ms.Bandi Bhanusri** a second-year student of Bachelor of Technology in Computer science and Engineering (Data Science), Gayatri Vidya Parishad College of Engineering, worked under my supervision during her internship period from **20th of May 2024 to 30th of June 2024** as a Summer Intern on full-time basis at NIT Warangal.

During the period of her internship program with us, she had been exposed to different processes in the domain of Machine Learning and Recommended Systems and was found diligent, hardworking and inquisitive. She had acquired knowledge of TensorFlow, a machine learning framework based on the TensorFlow and Keras library and completed various tasks across the different datasets. She have gone through an online course on Machine Learning. Her strong motivation and proactive attitude towards learning new concepts were evident and commendable.

I wish her all the best for her future endeavours.


Dr. Venkateswara Rao Kagita
Assistant Professor, CSE
Department of Computer Science & Engineering,
National Institute of Technology, Warangal
Telangana - INDIA - 506 004.

Acknowledgment

I would like to express my sincere gratitude to the **National Institute of Technology, Warangal**, for providing me the valuable opportunity to undertake this internship in the field of **Machine Learning and Recommender Systems**.

I am deeply thankful to **Dr. Venkateshwara Rao Kagita**, Assistant Professor, Department of CSE (AI & ML), NIT Warangal, for giving me this opportunity and for his constant support and encouragement throughout the internship.

I would also like to extend my heartfelt thanks to **Adamyra Sir** and **Shailendra Sir**, Ph.D. scholars under the guidance of Dr. Venkateshwara Rao Kagita, for their dedicated mentorship, timely feedback, and for guiding us patiently during each phase of the internship. Their insights and expertise in the field of machine learning and recommender systems played a key role in helping me understand the practical aspects of research and development.

This internship has been a highly enriching and enlightening experience, and I am truly grateful for the learning opportunity I received through this program.

Abstract

During this internship at NIT Warangal, we explored and implemented deep learning techniques to enhance recommender systems. Specifically, we developed and compared deep collaborative filtering models like DeepMF, NCF, Variational DeepMF, and Variational NCF — on the MovieLens 1M dataset. Among these, DeepMF achieved the best performance with an NDCG score of 0.94. Evaluation metrics such as MAE, R^2 , Precision, and Recall were used to assess model performance.

Additionally, we applied a hybrid Neural-SVM (NSVM) approach, where neural networks were used to extract features that were then passed to a support vector regressor. This combination further demonstrated the effectiveness of integrating deep learning with traditional ML techniques in recommender systems.

Introduction

I undertook a 40-day research internship at the National Institute of Technology (NIT), Warangal, under the guidance of Dr. Venkateshwara Rao Kagita, in the field of Machine Learning and Recommender Systems.

During this internship, I gained practical experience and deep insights into modern machine learning techniques, with a focus on deep learning-based collaborative filtering models, including Deep Matrix Factorization (DeepMF) and Neural Collaborative Filtering (NCF). I also explored the integration of Variational Autoencoders (VAEs) into recommender systems to enhance latent space structure and improve recommendation quality.

This report outlines the work I carried out, including the datasets used, model architectures, evaluation metrics, and results obtained. It also highlights the challenges faced and the knowledge I acquired throughout the internship.

Methodology

1.Literature Review & Learning Phase

- Started with a research paper provided by the mentor to understand the background.

Paper Summary: Deep Variational Models for Collaborative Filtering-Based Recommender Systems

As part of the internship, I was assigned the research paper titled "**Deep Variational Models for Collaborative Filtering-Based Recommender Systems**" by Jesús Bobadilla et al. This paper explores the integration of **variational autoencoders (VAEs)** into deep collaborative filtering models to enhance the quality of recommendations.

Traditional models such as **Deep Matrix Factorization (DeepMF)** and **Neural Collaborative Filtering (NCF)** have shown great promise in recommender systems by using deep neural networks to learn latent representations of users and items. However, these models lack **stochasticity**, which limits the structure, continuity, and robustness of the latent space — a critical aspect for generating meaningful recommendations.

To overcome this, the authors propose **Variational DeepMF (VDeepMF)** and **Variational NCF (VNCF)**, which incorporate VAE components into the latent space encoding process. Instead of augmenting the sparse user–item interaction data, the stochastic variation is injected **within the dense latent space** of the model. This helps in learning **regularized, complete, and continuous latent distributions** for users and items, thereby improving generalization and model robustness.

The proposed models were evaluated across **four benchmark datasets** (FilmTrust, MovieLens, MyAnimeList, and Netflix) and measured using multiple performance metrics like **MAE, MSE, R^2 , Precision, Recall, F1-score, NDCG, and Novelty**. Results showed that the variational approach **significantly improved performance** in small to medium datasets (like MovieLens and FilmTrust), especially when used with simpler models like DeepMF. However, on very large datasets such as Netflix, the performance gain was limited due to the introduction of noise from excessive stochasticity.

The main takeaways from the paper are:

- VAEs introduce **controlled randomness** that improves the latent space quality.
- **VDeepMF outperformed DeepMF** in multiple cases, especially in terms of novelty and NDCG.
- The proposed models reduce training time and enhance robustness without needing separate generative stages.
- **Variational enhancement** is most effective when applied to datasets that are sparse or less predictable.

This paper laid the theoretical foundation for the models I implemented during my internship, particularly the integration of stochastic latent representations in recommendation systems.

- Studied core machine learning concepts such as Support Vector Machines (SVM), regression, classification, and deep learning models.
- Understood recommendation systems, focusing on collaborative filtering techniques like matrix factorization.

2. Dataset Used

- The project utilized the **MovieLens 1M dataset**, which includes:
 - Over **1 million user–item–rating** interactions.
 - Metadata about users, movies, and ratings.

3. Model Implementation

As part of the internship, I implemented and evaluated four deep learning-based collaborative filtering models using the MovieLens 1M dataset, which contains over one million user-item interactions. The goal was to predict user preferences and evaluate model performance using metrics such as NDCG, MAE, R^2 , Precision, and Recall. Below is a brief explanation of each model:

➤ DeepMF (Deep Matrix Factorization)

DeepMF is a deep learning adaptation of traditional matrix factorization. It replaces the linear latent factor models with a multi-layer neural network to learn user and item embeddings. The model takes user and item IDs as inputs and maps them through embedding layers followed by dense layers. The final output is the predicted rating.

Strengths: Efficient, simple, and provides better performance than shallow MF models.

Use Case: Effective in capturing user-item interaction patterns in structured data.

➤ NCF (Neural Collaborative Filtering)

NCF extends DeepMF by replacing the dot product used in matrix factorization with a multi-layer perceptron (MLP) that learns the interaction function from data. It combines the embeddings of users and items and passes them through multiple non-linear layers to model complex relationships.

Strengths: Learns non-linear interaction patterns and typically performs better than DeepMF.

Use Case: Suitable when deeper user-item interaction modeling is required.

➤ Variational DeepMF

This model builds upon DeepMF by integrating variational inference through a Variational Autoencoder (VAE) framework. Instead of deterministic latent embeddings, it learns probabilistic latent variables that introduce stochasticity in the latent space. This allows for better generalization and robustness in sparse settings.

Strengths: Structured and continuous latent space; better at capturing uncertainty.

Use Case: Ideal for sparse datasets or where regularization is crucial.

➤ Variational NCF

Variational NCF combines the strengths of both NCF and VAEs. It models user-item interactions using a deep MLP as in NCF, but replaces deterministic layers with stochastic latent variables. This hybrid structure helps in learning richer and more expressive user and item representations.

Strengths: Robust modeling of uncertainty; better performance in moderate data sparsity.

Use Case: Suitable when modeling complex interactions while incorporating uncertainty.

➤ Hybrid NSVM Model

- Built a neural network to extract high-level features from user-item pairs.
- Passed these features to an SVM (Support Vector Regressor) for rating prediction.
- Trained both the neural net and SVM iteratively.
- Achieved satisfactory MSE values on test data, showing the effectiveness of the hybrid method.

4.Evaluation Summary

All models were evaluated using the following performance metrics:

- NDCG (Normalized Discounted Cumulative Gain): Measures the quality of ranked recommendations.
- MAE (Mean Absolute Error): Measures the average absolute error between predicted and actual ratings.
- R^2 Score: Measures how well the variance in ratings is explained by the model.
- Precision and Recall: Evaluate the relevance of recommended items.

Among the models, DeepMF achieved the highest NDCG score of 0.94, making it the most effective in ranking-based evaluation. Variational models showed promising performance in capturing uncertainty and improving latent space structure.

CODE IMPLEMENTATION

```
import math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from keras.models import Model, Sequential
from keras.layers import Embedding, Flatten, Input, Dropout, Dense, Concatenate, Dot, Lambda
from keras.optimizers import Adam
from keras import backend as K

from sklearn.metrics import precision_score, recall_score, ndcg_score, mean_absolute_error
from sklearn.model_selection import train_test_split

dataset = 'ml1m' # MovieLens 1M dataset
latent_dim = 5
like_threshold = 4
steps_per_epoch = None

deepmf_epochs = 10
ncf_epochs = 10
neumf_epochs = 10
vdeepmf_epochs = 6
vncf_epochs=10

# Load the dataset
df = pd.read_csv('/ml1m.csv', delimiter=',')

num_users = df.user.max() + 1
num_items = df.item.max() + 1

X = df[['user', 'item']].to_numpy()
y = df[['rating']].to_numpy()

#Dataset split into train and test partitions.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

X_train = [X_train[:, 0], X_train[:, 1]]
X_test = [X_test[:, 0], X_test[:, 1]]
```

✓ DeepMF Model

```
# DeepMF Model
user_input = Input(shape=[1])
user_embedding = Embedding(num_users, latent_dim)(user_input)
user_vec = Flatten()(user_embedding)
```



```

item_input = Input(shape=[1])
item_embedding = Embedding(num_items, latent_dim)(item_input)
item_vec = Flatten()(item_embedding)

dot = Dot(axes=1)([item_vec, user_vec])

DeepMF = Model([user_input, item_input], dot)

with tf.device('/GPU:1'):
    DeepMF.compile(optimizer='adam', metrics=['mae'], loss='mean_squared_error')
    DeepMF.summary()

    deepmf_report = DeepMF.fit(X_train, y_train, validation_data=(X_test, y_test), epochs

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 1)	0	-
input_layer (InputLayer)	(None, 1)	0	-
embedding_1 (Embedding)	(None, 1, 5)	18,530	input_layer_1[0]...
embedding (Embedding)	(None, 1, 5)	30,200	input_layer[0][0]
flatten_1 (Flatten)	(None, 5)	0	embedding_1[0][0]
flatten (Flatten)	(None, 5)	0	embedding[0][0]
dot (Dot)	(None, 1)	0	flatten_1[0][0], flatten[0][0]

Total params: 48,730 (190.35 KB)
 Trainable params: 48,730 (190.35 KB)
 Non-trainable params: 0 (0.00 B)

Epoch 1/10
 25006/25006 ————— 68s 3ms/step - loss: 8.4971 - mae: 2.4981 - val_loss
 Epoch 2/10
 25006/25006 ————— 83s 3ms/step - loss: 0.9571 - mae: 0.7659 - val_loss
 Epoch 3/10
 25006/25006 ————— 80s 3ms/step - loss: 0.8732 - mae: 0.7358 - val_loss
 Epoch 4/10
 25006/25006 ————— 84s 3ms/step - loss: 0.8578 - mae: 0.7306 - val_loss
 Epoch 5/10
 25006/25006 ————— 67s 3ms/step - loss: 0.8494 - mae: 0.7264 - val_loss
 Epoch 6/10
 25006/25006 ————— 67s 3ms/step - loss: 0.8352 - mae: 0.7197 - val_loss
 Epoch 7/10
 25006/25006 ————— 81s 3ms/step - loss: 0.8151 - mae: 0.7105 - val_loss
 Epoch 8/10
 25006/25006 ————— 65s 3ms/step - loss: 0.7931 - mae: 0.7001 - val_loss
 Epoch 9/10
 25006/25006 ————— 64s 3ms/step - loss: 0.7796 - mae: 0.6931 - val_loss

✓ NCF Model

```
# NCF Model
item_input = Input(shape=[1], name='item-input')
item_embedding = Embedding(num_items, latent_dim, name='item-embedding')(item_input)
item_vec = Flatten(name='item-flatten')(item_embedding)

user_input = Input(shape=[1], name='user-input')
user_embedding = Embedding(num_users, latent_dim, name='user-embedding')(user_input)
user_vec = Flatten(name='user-flatten')(user_embedding)

concat = Concatenate(axis=1, name='item-user-concat')([item_vec, user_vec])
fc_1 = Dense(70, name='fc-1', activation='relu')(concat)
fc_1_dropout = Dropout(0.5, name='fc-1-dropout')(fc_1)
fc_2 = Dense(30, name='fc-2', activation='relu')(fc_1_dropout)
fc_2_dropout = Dropout(0.4, name='fc-2-dropout')(fc_2)
fc_3 = Dense(1, name='fc-3', activation='relu')(fc_2_dropout)

NCF = Model([user_input, item_input], fc_3)

with tf.device('/GPU:1'):
    NCF.compile(optimizer='adam', metrics=['mae'], loss='mean_squared_error')
    NCF.summary()

    ncf_report = NCF.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=ncf_e
```

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
item-input (InputLayer)	(None, 1)	0	-
user-input (InputLayer)	(None, 1)	0	-
item-embedding (Embedding)	(None, 1, 5)	18,530	item-input[0][0]
user-embedding (Embedding)	(None, 1, 5)	30,200	user-input[0][0]
item-flatten (Flatten)	(None, 5)	0	item-embedding[0...
user-flatten (Flatten)	(None, 5)	0	user-embedding[0...
item-user-concat (Concatenate)	(None, 10)	0	item-flatten[0][... user-flatten[0][...
fc-1 (Dense)	(None, 70)	770	item-user-concat...
fc-1-dropout (Dropout)	(None, 70)	0	fc-1[0][0]
fc-2 (Dense)	(None, 30)	2,130	fc-1-dropout[0][...
fc-2-dropout (Dropout)	(None, 30)	0	fc-2[0][0]
fc-3 (Dense)	(None, 1)	31	fc-2-dropout[0][...

Total params: 51,661 (201.80 KB)

Trainable params: 51,661 (201.80 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

25006/25006 ————— 83s 3ms/step - loss: 1.3833 - mae: 0.9163 - val_1

Epoch 2/10

25006/25006 ————— 91s 4ms/step - loss: 0.8489 - mae: 0.7295 - val_1

Epoch 3/10

25006/25006 ————— 133s 3ms/step - loss: 0.8258 - mae: 0.7179 - val_1

Epoch 4/10

25006/25006 ————— 82s 3ms/step - loss: 0.8110 - mae: 0.7102 - val_1

Epoch 5/10

25006/25006 ————— 147s 3ms/step - loss: 0.8020 - mae: 0.7058 - val_1

Epoch 6/10

25006/25006 ————— 79s 3ms/step - loss: 0.7929 - mae: 0.7005 - val_1

Epoch 7/10

25006/25006 ————— 81s 3ms/step - loss: 0.7872 - mae: 0.6977 - val_1

Epoch 8/10

25006/25006 ————— 78s 3ms/step - loss: 0.7808 - mae: 0.6946 - val_1

Epoch 9/10

✓ Variational DeepMF Model

```
# Variational DeepMF Model
batch_size = 32

def sampling(args):
    z_mean, z_var = args
    epsilon = tf.random.normal(shape=tf.shape(z_mean), mean=0.0, stddev=1.0)

    return z_mean + tf.exp(z_var) * epsilon

user_input = Input(shape=[1])
user_embedding = Embedding(num_users, latent_dim)(user_input)
user_embedding_mean = Dense(latent_dim)(user_embedding)
user_embedding_var = Dense(latent_dim)(user_embedding)
user_embedding_z = Lambda(sampling, output_shape=(1, latent_dim))([user_embedding_mean, u

user_vec = Flatten()(user_embedding_z)

item_input = Input(shape=[1])
item_embedding = Embedding(num_items, latent_dim)(item_input)
item_embedding_mean = Dense(latent_dim)(item_embedding)
item_embedding_var = Dense(latent_dim)(item_embedding)
item_embedding_z = Lambda(sampling, output_shape=(1, latent_dim))([item_embedding_mean, i
item_vec = Flatten()(item_embedding_z)

dot = Dot(axes=1)([item_vec, user_vec])

VDeepMF = Model([user_input, item_input], dot)

with tf.device('/GPU:1'):
    VDeepMF.compile(optimizer='adam', metrics=['mae'], loss='mean_squared_error')
    VDeepMF.summary()

vdeepmf_report = VDeepMF.fit(X_train, y_train, validation_data=(X_test, y_test), epoc
```

Model: "functional_7"

Layer (type)	Output Shape	Param #	Connected to
input_layer_12 (InputLayer)	(None, 1)	0	-
input_layer_11 (InputLayer)	(None, 1)	0	-
embedding_12 (Embedding)	(None, 1, 5)	18,530	input_layer_12[0...]
embedding_11 (Embedding)	(None, 1, 5)	30,200	input_layer_11[0...]
dense_20 (Dense)	(None, 1, 5)	30	embedding_12[0][...]
dense_21 (Dense)	(None, 1, 5)	30	embedding_12[0][...]
dense_18 (Dense)	(None, 1, 5)	30	embedding_11[0][...]
dense_19 (Dense)	(None, 1, 5)	30	embedding_11[0][...]
lambda_10 (Lambda)	(None, 1, 5)	0	dense_20[0][0], dense_21[0][0]
lambda_9 (Lambda)	(None, 1, 5)	0	dense_18[0][0], dense_19[0][0]
flatten_11 (Flatten)	(None, 5)	0	lambda_10[0][0]
flatten_10 (Flatten)	(None, 5)	0	lambda_9[0][0]
dot_5 (Dot)	(None, 1)	0	flatten_11[0][0], flatten_10[0][0]

Total params: 48,850 (190.82 KB)

Trainable params: 48,850 (190.82 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/6

25006/25006 ————— 98s 4ms/step - loss: 4.0214 - mae: 1.3967 - val_loss

Epoch 2/6

25006/25006 ————— 102s 4ms/step - loss: 0.8004 - mae: 0.7049 - val_loss

Epoch 3/6

25006/25006 ————— 134s 4ms/step - loss: 0.7583 - mae: 0.6838 - val_loss

Epoch 4/6

25006/25006 ————— 85s 3ms/step - loss: 0.7377 - mae: 0.6734 - val_loss

Epoch 5/6

+ Code

+ Text

✓ Variational NCF Model

```
# Variational NCF Model
```

```
batch_size = 32
```

```

def sampling(args):
    z_mean, z_var = args
    epsilon = tf.random.normal(shape=tf.shape(z_mean), mean=0.0, stddev=1.0)
    return z_mean + tf.exp(z_var) * epsilon

user_input = Input(shape=[1])
user_embedding = Embedding(num_users, latent_dim)(user_input)
user_embedding_mean = Dense(latent_dim)(user_embedding)
user_embedding_var = Dense(latent_dim)(user_embedding)
user_embedding_z = Lambda(sampling, output_shape=(1, latent_dim))([user_embedding_mean, u

user_vec = Flatten()(user_embedding_z)

item_input = Input(shape=[1])
item_embedding = Embedding(num_items + 1, latent_dim)(item_input)
item_embedding_mean = Dense(latent_dim)(item_embedding)
item_embedding_var = Dense(latent_dim)(item_embedding)
item_embedding_z = Lambda(sampling, output_shape=(1, latent_dim))([item_embedding_mean, i
item_vec = Flatten()(item_embedding_z)

concat = Concatenate(axis=1)([item_vec, user_vec])

fc_1 = Dense(80, name='fc-1', activation='relu')(concat)
fc_1_dropout = Dropout(0.6, name='fc-1-dropout')(fc_1)
fc_2 = Dense(25, name='fc-2', activation='relu')(fc_1_dropout)
fc_2_dropout = Dropout(0.4, name='fc-2-dropout')(fc_2)
fc_3 = Dense(1, name='fc-3', activation='relu')(fc_2_dropout)

VNCF = Model([user_input, item_input], fc_3)

with tf.device('/GPU:1'):
    VNCF.compile(optimizer='adam', metrics=['mae'], loss='mean_squared_error')
    VNCF.summary()

    vncf_report = VNCF.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=vnc

```

Model: "functional_10"

Layer (type)	Output Shape	Param #	Connected to
input_layer_19 (InputLayer)	(None, 1)	0	-
input_layer_18 (InputLayer)	(None, 1)	0	-
embedding_19 (Embedding)	(None, 1, 5)	18,535	input_layer_19[0...]
embedding_18 (Embedding)	(None, 1, 5)	30,200	input_layer_18[0...]
dense_34 (Dense)	(None, 1, 5)	30	embedding_19[0][...]
dense_35 (Dense)	(None, 1, 5)	30	embedding_19[0][...]
dense_32 (Dense)	(None, 1, 5)	30	embedding_18[0][...]
dense_33 (Dense)	(None, 1, 5)	30	embedding_18[0][...]
lambda_17 (Lambda)	(None, 1, 5)	0	dense_34[0][0], dense_35[0][0]
lambda_16 (Lambda)	(None, 1, 5)	0	dense_32[0][0], dense_33[0][0]
flatten_17 (Flatten)	(None, 5)	0	lambda_17[0][0]
flatten_16 (Flatten)	(None, 5)	0	lambda_16[0][0]
concatenate_2 (Concatenate)	(None, 10)	0	flatten_17[0][0], flatten_16[0][0]
fc-1 (Dense)	(None, 80)	880	concatenate_2[0]...
fc-1-dropout (Dropout)	(None, 80)	0	fc-1[0][0]
fc-2 (Dense)	(None, 25)	2,025	fc-1-dropout[0][...]
fc-2-dropout (Dropout)	(None, 25)	0	fc-2[0][0]
fc-3 (Dense)	(None, 1)	26	fc-2-dropout[0][...]

Total params: 51,786 (202.29 KB)

Trainable params: 51,786 (202.29 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

25006/25006 ————— 105s 4ms/step - loss: 1.4452 - mae: 0.9489 - val_

Epoch 2/10

25006/25006 ————— 96s 4ms/step - loss: 0.8661 - mae: 0.7356 - val_1

Epoch 3/10

25006/25006 ————— 97s 4ms/step - loss: 0.8433 - mae: 0.7233 - val_1

Epoch 4/10

21/05/2025, 07:02

neural.ipynb - Colab

```
25006/25006 ————— 145s 4ms/step - loss: 0.8329 - mae: 0.7187 - val_
Epoch 5/10
25006/25006 ————— 99s 4ms/step - loss: 0.8248 - mae: 0.7146 - val_l
Epoch 6/10
25006/25006 ————— 152s 4ms/step - loss: 0.8160 - mae: 0.7101 - val_
Epoch 7/10
25006/25006 ————— 100s 4ms/step - loss: 0.8126 - mae: 0.7093 - val_
Epoch 8/10
25006/25006 ————— 103s 4ms/step - loss: 0.8033 - mae: 0.7049 - val_
- ...
```

✓ Evaluation

```
# Evaluation
methods = ['vdeepmf', 'deepmf', 'vncf', 'ncf']
methods = ['deepmf']
preds = pd.DataFrame()

preds['user'] = X_test[0]
preds['item'] = X_test[1]
preds['y_test'] = y_test
y_true = preds['y_test'].to_numpy()

# Store predictions of the baselines
preds['deepmf'] = DeepMF.predict(X_test)
preds['ncf'] = NCF.predict(X_test)

# Variational models predictions
n_repeats = 10
predictions = None

for i in range(n_repeats):
    if i == 0:
        predictions = VDeepMF.predict(X_test)
    else:
        predictions = np.append(predictions, VDeepMF.predict(X_test), axis=1)

preds['vdeepmf'] = np.mean(predictions, axis=1)
predictions = None

for i in range(n_repeats):
    if i == 0:
        predictions = VNCF.predict(X_test)
    else:
        predictions = np.append(predictions, VNCF.predict(X_test), axis=1)

preds['vncf'] = np.mean(predictions, axis=1)

print('MAE:')
for m in methods:
    print('-', m, ':', mean_absolute_error(preds['y_test'], preds[m]))
```



```

print('MSD:')
for m in methods:
    print('-', m, ':', mean_squared_error(preds['y_test'], preds[m]))

print('R2:')
for m in methods:
    print('-', m, ':', r2_score(preds['y_test'], preds[m]))

y_true = preds['y_test'].apply(lambda x: 1 if x > like_threshold else 0).to_numpy()
preds_bin = preds.drop('y_test', axis=1).applymap(lambda x: 1 if x > like_threshold else 0)
print('Precision:')
for m in methods:
    print('-', m, ':', precision_score(y_true, preds_bin[m]))

print('Recall:')
for m in methods:
    print('-', m, ':', recall_score(y_true, preds_bin[m]))

print('NDCG:')
for m in methods:
    print('-', m, ':', ndcg_score([y_true], [preds[m]]))

```

 6252/6252 ————— 9s 1ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 10s 2ms/step
 6252/6252 ————— 10s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 13s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 12s 2ms/step
 6252/6252 ————— 13s 2ms/step
 6252/6252 ————— 12s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 12s 2ms/step
 6252/6252 ————— 12s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 11s 2ms/step
 6252/6252 ————— 12s 2ms/step
 6252/6252 ————— 11s 2ms/step
 MAE:
 - deepmf : 0.6991248490206347
 MSD:
 - deepmf : 0.793968825572721
 R2:
 - deepmf : 0.36519653729942947
 <ipython-input-27-a91fb45d0c77>:51: FutureWarning: DataFrame.applymap has been deprec
 preds_bin = preds.drop('y_test', axis=1).applymap(lambda x: 1 if x > like_threshold
 Precision:
 - deepmf : 0.47168774051676743
 Recall:
 - deepmf : 0.6258205689277899
 NDCG:

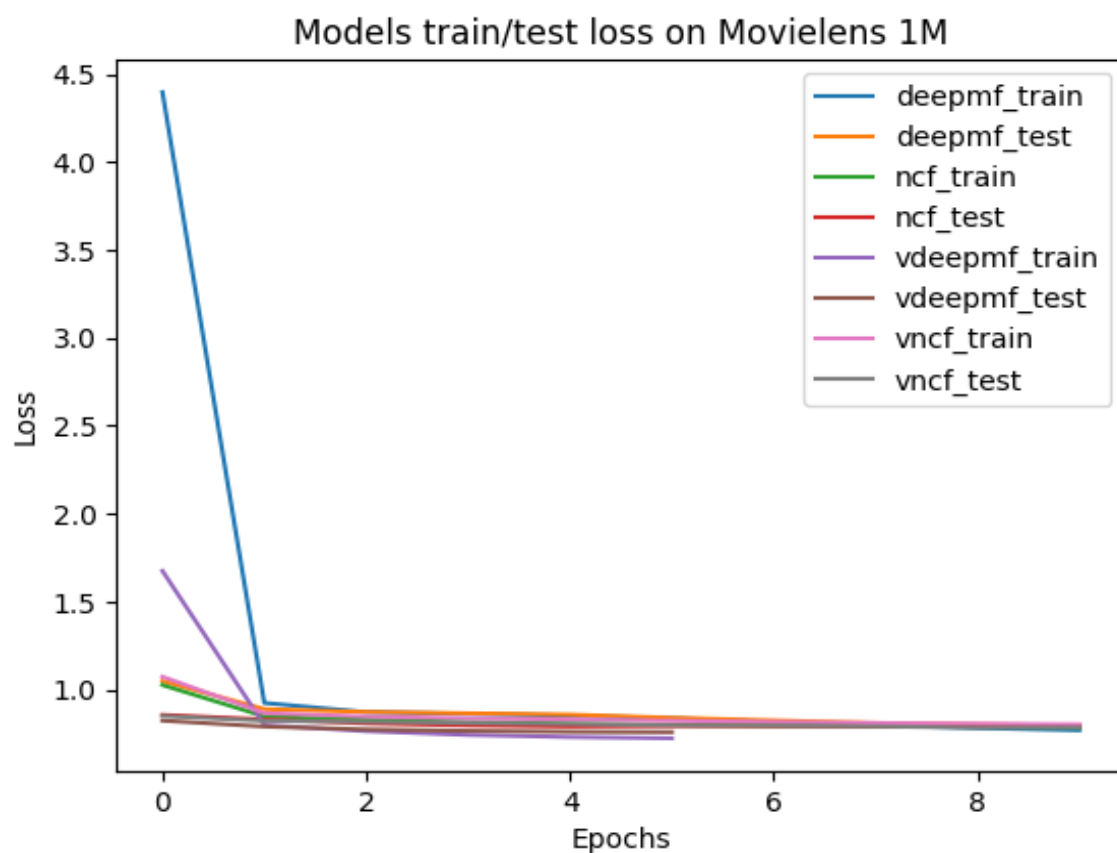
```
plt.plot(deepmf_report.history['loss'], label='deepmf_train')
plt.plot(deepmf_report.history['val_loss'], label='deepmf_test')

plt.plot(ncf_report.history['loss'], label='ncf_train')
plt.plot(ncf_report.history['val_loss'], label='ncf_test')

plt.plot(vdeepmf_report.history['loss'], label='vdeepmf_train')
plt.plot(vdeepmf_report.history['val_loss'], label='vdeepmf_test')

plt.plot(vncf_report.history['loss'], label='vncf_train')
plt.plot(vncf_report.history['val_loss'], label='vncf_test')

plt.title('Models train/test loss on Movielens 1M')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc='best')
plt.show()
```



▼ Recommendations

```
[ ] def recommend_movies_for_user_filtered(model, user_id, df, num_items, top_n=10):  
    # Items user has rated  
    itemsRated = df[df.user == user_id].item.values  
  
    # Candidate items = all items except rated ones  
    candidate_items = np.setdiff1d(np.arange(num_items), itemsRated)  
  
    user_array = np.full_like(candidate_items, user_id)  
    preds = model.predict([user_array, candidate_items], verbose=0).flatten()  
  
    top_indices = preds.argsort()[-top_n:][::-1]  
    recommended_items = candidate_items[top_indices]  
    return recommended_items  
  
# Usage:  
recommended_movie_ids = recommend_movies_for_user_filtered(DeepMF, user_id=5, df=df, num_items=num_items, top_n=10)  
print(recommended_movie_ids)
```

 [3460 892 1000 2938 3107 2875 3493 1952 3576 3261]

Movie Recommendation Output

- After training, the models were able to recommend personalized movie suggestions to users based on their historical preferences.
- The system ensures efficient and accurate delivery of recommendations, improving user satisfaction.

Summary and Conclusions

This internship at NIT Warangal provided a comprehensive exploration into deep learning approaches for recommender systems, with a focus on collaborative filtering techniques. Leveraging the MovieLens 1M dataset, we implemented and analyzed a range of models including Deep Matrix Factorization (DeepMF), Neural Collaborative Filtering (NCF), and their variational enhancements—Variational DeepMF and Variational NCF. These models were evaluated using metrics such as NDCG, MAE, R^2 , Precision, and Recall to assess their ability to accurately predict and rank user preferences.

Among the models, **DeepMF achieved the highest performance with an NDCG score of 0.94**, indicating strong ranking capabilities. Variational models demonstrated improved generalization and robustness, particularly useful for sparse data scenarios, due to their structured and continuous latent space enabled by variational inference. Furthermore, a hybrid **Neural-SVM (NSVM)** model was developed, which combined deep learning-based feature extraction with support vector regression. This hybrid approach further confirmed the potential of integrating deep and traditional machine learning techniques for recommendation tasks.

In conclusion, the internship not only deepened our understanding of advanced recommender systems but also emphasized the importance of choosing models based on data characteristics and application requirements. It strengthened our practical skills in deep learning, variational modeling, hybrid architectures, and performance evaluation, thereby equipping us with the expertise needed to address real-world challenges in personalized recommendation systems.

References

1. Research paper on Deep Variational Models for Collaborative Filtering-based Recommender Systems

<https://arxiv.org/pdf/2107.12677>

2. Matrix Factorization approach from collaborative Filtering .

<https://youtu.be/ZspR5PZemcs?si=HZxtjP9P3CM0eNiu>

https://youtu.be/JJnIR_S6dg0?si=oM6QHng5B7LXxzI7

Code implementation:

<https://github.com/Bhanusri08/Deep-Variational-Models-for-Collaborative-Filtering-based-Recommender-Systems>